

Job Pipeline — Architecture Overview

Purpose

An automated job search pipeline that fetches Engineering Manager and Director-level roles from 30+ company career sites daily, scores each against a candidate profile using an LLM, and surfaces the best matches in a web UI. Cover letters can be generated and exported on demand.

High-Level Architecture



Component Breakdown

Web Layer

File	Role
web/server.py	FastAPI app — all API routes, background task dispatch
web/tasks.py	In-process task runner — runs pipeline steps in background threads, streams logs to UI
web/auth.py	API key middleware (x-api-key header, checked against WEB_API_KEY env var)
web/static/app.js	Single-page app — job list, filters, detail pane, task drawer
web/static/index.html	SPA shell
web/static/style.css	Styles

Pipeline Modules

File	Role
pipeline/fetcher.py	Fetches jobs from Greenhouse, Ashby, Lever, and custom APIs (Netflix, Zillow, Amazon, LinkedIn, Microsoft, Uber). Handles pagination, title/location filtering.
pipeline/playwright_fetcher.py	Browser-based fetchers for Google, Apple, Meta, Walmart — sites that render jobs via JavaScript and can't be scraped with HTTP alone. Each company gets a fresh Chromium instance.
pipeline/scorer.py	JobScorer — single LLM call, returns a 1-10 score with summary, strengths, and gaps. Location penalties applied based on preferences.yaml.
pipeline/evaluator.py	JobEvaluator — two-call LLM evaluation: first extracts requirements from the job description, then evaluates each against the candidate profile. Returns per-requirement fit (Strong / Partial / Gap) and resume suggestions. On-demand only.
pipeline/generator.py	ContentGenerator — generates a cover letter in Markdown via LLM. Saves to output/{Company}_{job_id}/cover_letter.md. On-demand only.
pipeline/store.py	JobStore — SQLite wrapper. Deduplicates by (company, job_id). Stores scores, match analysis, status, and dates. Supports run history.
pipeline/profile.py	ProfileLoader — loads profile/resume.md, profile/experience.md, and optional Google Docs links into a structured dict used for all LLM prompts.
pipeline/llm.py	LLMProvider — unified interface for Gemini, Anthropic, OpenAI, and Ollama. Provider selected from preferences.yaml.
pipeline/detect_ats.py	Auto-detects ATS platform for a company name (used by Settings > Add Company).
pipeline/alerter.py	Gmail SMTP alerter — sends digest emails for high-scoring new jobs.
pipeline/ingester.py pipeline/checkpoint.py	Ingests supplementary documents (PDFs, DOCX) into the profile. Checkpoint state for long CLI runs.

Config Files

File	Role
config/companies.yaml	List of companies with ATS type and board slug
config/preferences.yaml	Match threshold, LLM provider, location preferences, title keywords
profile/resume.md	Candidate resume (plain Markdown)
profile/experience.md	Extended experience narrative
profile/cover_letter_style_guide.md	Style guidance injected into cover letter generation prompts

Data Flow

Source + Score Run

```
User clicks "Run" in UI
|
POST /api/pipeline/run
|
Background task starts (_do_run)
|
fetch_all_companies()
- HTTP fetchers: Greenhouse / Ashby / Lever / custom APIs
- Playwright fetchers: Google / Apple / Meta / Walmart
|
For each job returned:
  store.upsert_job(job)      -- deduplicate; track date_posted / date_last_sourced
  |
For each NEW job:
  scorer.score(job, profile, prefs)
  - LLM prompt with profile + job description
  - Returns score 1-10, summary, strengths, gaps
  store.set_match_score(...)
  If score >= threshold: status = "alerted"
  Else:                  status = "skipped"
  |
Task log streamed to UI task drawer (polled every 1.5s)
|
UI reloads job list on completion
```

On-Demand Actions

```
Generate Cover Letter      POST /api/jobs/{company}/{job_id}/generate-cover-letter
                           -> ContentGenerator -> cover_letter.md

Export PDF                 POST /api/jobs/{company}/{job_id}/export-cover-letter-
pdf
```

```

                                -> pandoc + typst -> cover_letter.pdf -> auto-
download

Deep Evaluation                POST /api/jobs/{company}/{job_id}/analyze
                                -> JobEvaluator (2 LLM calls)
                                -> per-requirement fit + resume suggestions

Rescore                        POST /api/jobs/{company}/{job_id}/rescore
                                -> JobScorer -> updates score in DB

```

Database Schema

SQLite at jobs.db (on Fly.io: /data/jobs.db on persistent volume).

jobs table — primary key (company, job_id)

Column	Type	Notes
company	TEXT	Company name
job_id	TEXT	ATS job ID
title	TEXT	
location	TEXT	
description	TEXT	Full job description
apply_url	TEXT	Direct application link
status	TEXT	new / alerted / approved / applied / skipped / interview- ing / rejected / offer / inter- esting
match_score	INTEGER	Adjusted score (1-10) after lo- cation penalty
match_summary	TEXT	LLM summary
match_strengths	TEXT	JSON array
match_gaps	TEXT	JSON array
match_requirements	TEXT	JSON array (deep evaluation)
match_resume_suggestions	TEXT	JSON array
date_posted	TEXT	ISO date from ATS (COALESCE — only set once)
date_last_sourced	TEXT	ISO date — updated on every re-fetch
date_seen	TEXT	First time job appeared

runs table — pipeline run history

Column	Notes
id	Auto-increment
started_at / ended_at	Timestamps
action	source / source_and_score / score / rescore
group_type	http / playwright / all
companies_count	Number targeted
jobs_fetched / jobs_new / jobs_scored	Counters
status	running / done / error
error_msg	Set on failure

ATS Coverage

ATS	Companies	Fetch Method
Greenhouse	DoorDash, Dropbox, AirBnB, Stripe, Anthropic, ScaleAI, Duolingo, Lyft, StubHub, Instacart, Figma, Databricks, Pinterest, MongoDB, Twilio, Roblox, HubSpot, CoreWeave	HTTP (public board API)
Ashby	OpenAI, Cohere, Ramp, Notion	HTTP (public board API)
Lever	Spotify	HTTP (public board API)
Custom HTTP	Netflix, Zillow, Amazon, LinkedIn, Microsoft, Uber	Company-specific API endpoints
Playwright	Google, Apple, Meta, Walmart	Headless Chromium (JavaScript-rendered pages)

Docker

Base Image

python:3.12-slim — minimal Debian-based Python image.

System Dependencies (apt)

- chromium + chromium-driver — headless browser for Playwright fetchers
- Graphics/audio libs (libnss3, libatk*, libcups2, etc.) — required by Chromium
- pandoc — Markdown to PDF conversion
- wget + xz-utils — used to download the typst binary

Typst

Downloaded at build time from the GitHub release: `typst-x86_64-unknown-linux-musl v0.13.1`
— PDF engine used by pandoc. Installed to `/usr/local/bin/typst`.

Python Dependencies

Installed via `pip install -r requirements.txt`:

Package	Purpose
<code>fastapi + uvicorn</code>	Web server
<code>google-genai</code>	Gemini LLM provider
<code>anthropic</code>	Claude LLM provider
<code>openai</code>	OpenAI + Ollama (OpenAI-compatible API)
<code>playwright</code>	Browser automation
<code>requests + beautifulsoup4</code>	HTTP fetching and HTML parsing
<code>pyyaml</code>	Config file parsing
<code>python-docx + pypdf</code>	Document ingestion
<code>click</code>	CLI interface

Build Layers (order optimised for cache)

1. `apt install` (system deps) -- rarely changes, cached
2. `wget typst binary` -- rarely changes, cached
3. `WORKDIR /app`
4. `COPY requirements.txt`
5. `pip install` -- cached unless requirements change
6. `playwright install chromium` -- cached
7. `COPY . .` -- invalidated on every code change
8. `mkdir /data`

Startup Command

```
uvicorn web.server:app --host 0.0.0.0 --port 8000
```

0.0.0.0 is required so Fly.io's proxy can route external traffic into the container.

Fly.io Configuration

App: `job-pipeline` **URL:** `https://job-pipeline.fly.dev` **Region:** `ewr` (Newark — closest to New York)

VM

```
[[vm]]
  cpu_kind = 'performance'
  cpus     = 1
  memory   = '2gb'
```

Performance CPU class is used because Playwright/Chromium is CPU-intensive. 2 GB RAM accommodates Chromium instances (each ~300-400 MB).

Persistent Volume

```
[[mounts]]
  source      = 'job_data'
  destination = '/data'
```

A persistent Fly volume mounted at /data. Contains:

Path	Contents
/data/jobs.db	SQLite database
/data/config/	companies.yaml, preferences.yaml (synced from image on startup)
/data/profile/	resume.md, experience.md, cover letter style guide
/data/output/	Generated cover letters and PDFs

Config files are versioned in the Docker image and copied into the volume on every startup — so deploying a new image always updates config, while jobs.db and generated files are preserved across deploys.

Auto-start / Auto-stop

```
auto_stop_machines = 'stop'
auto_start_machines = true
min_machines_running = 0
```

The machine stops when idle and starts automatically on the first incoming request. Cold start is roughly 5-10 seconds. This keeps costs near zero when the app is not in use.

HTTPS

```
force_https = true
```

All HTTP traffic is redirected to HTTPS by the Fly proxy.

Environment Variables (set as Fly secrets)

Variable	Purpose
WEB_API_KEY	Protects all API endpoints — required in x-api-key header
GEMINI_API_KEY	Gemini LLM provider
ANTHROPIC_API_KEY	Anthropic/Claude LLM provider
OPENAI_API_KEY	OpenAI LLM provider

Deploy Process

There is no CI/CD pipeline — deploys are manual from the local machine.

deploy.sh

1. Check for running tasks on the live app via GET /api/tasks
 - If a pipeline job is in progress, warn and ask before proceeding
 - Prevents killing a scoring run mid-flight
2. Run: fly deploy
 - Packages local directory into a Docker image
 - Builds on Fly's Depot builder (remote build)
 - Pushes image to Fly's registry
 - Rolls out to the machine with a health check

Typical deploy sequence

```
git add <files>
git commit -m "... "
git push origin main      # sync to GitHub
./deploy.sh              # deploy to Fly.io
```

Note: fly deploy builds from the **local directory**, not from GitHub. GitHub is used as a backup/source of truth only — it does not trigger deploys.

Future: GitHub Actions CI/CD (backlog)

The intended future state is:

```
git push -> GitHub Actions -> run tests -> fly deploy
```

This would ensure only passing code ships, and remove the need for manual deploys.

GitHub Repository

Repo: github.com/saurabh1011/job-pipeline (private) **Branch:** main (single long-lived branch)

No CI/CD is configured yet. Pushes to main do not trigger any automated action.

Local Development

```
# Start the server
python3 -m uvicorn web.server:app --reload --port 8000

# Run tests
python3 -m pytest tests/ -q

# Deploy
./deploy.sh
```

Environment variables needed locally: - WEB_API_KEY — set in ~/.zshrc - GEMINI_API_KEY (or other LLM key) — set in ~/.zshrc - JOB_API_KEY — required by deploy.sh to check for running tasks before deploying

Backlog

Item	Priority
Python 3.9 upgrade (local dev is 3.9 — EOL Oct 2025)	Urgent
GitHub Actions CI/CD (auto-deploy on push to main)	High
Daily scheduling (run sourcing + scoring once per day)	High
Multi-tenancy (multiple user profiles)	Medium
iOS PWA	Medium
Date field backfill for existing jobs	Low